Two-Stage Multi-Objective Genetic Programming with Archive for Uncertain Capacitated Arc Routing Problem

Shaolin Wang
shaolin.wang@ecs.vuw.ac.nz
Victoria University of Wellington
Wellington, New Zealand

Yi Mei
yi.mei@ecs.vuw.ac.nz
Victoria University of Wellington
Wellington, New Zealand

Mengjie Zhang
mengjie.zhang@ecs.vuw.ac.nz
Victoria University of Wellington
Wellington, New Zealand

ABSTRACT

Genetic Programming Hyper-Heuristic (GPHH) is a promising technique to automatically evolve effective routing policies to handle the uncertain environment in the Uncertain Capacitated Arc Routing Problem (UCARP). Previous studies mainly focus on the effectiveness of the evolved routing policies, but the size is ignored. This paper aims to develop new GPHH methods to optimise the effectiveness and the size simultaneously. There are two challenges. First, it is much easier for GP to generate small but ineffective individuals than effective ones, thus the search can be easily stuck with small but ineffective individuals. Second, the effectiveness evaluation in GPHH is stochastic, making it challenging to identify and retain effective individuals. To address these issues, we develop a Two-Stage Multi-Objective GP algorithm with Archive (TSNSGPII-a). The two-stage framework addresses the bias towards the size. The external archive stores potentially effective individuals that may be lost during the evolution, and reuses them to generate offspring. The experimental results show that TSNSGPII-a can obtain significantly better routing policies than the existing state-of-the-art approaches in terms of both effectiveness and size. If selecting the most effective routing policy from the Pareto front, TSNSGPII-a can obtain significantly smaller routing policies with statistically comparable or significantly better effectiveness.

CCS CONCEPTS

• **Computing methodologies** → *Planning under uncertainty*;

KEYWORDS

Genetic Programming, Hyper-Heuristic, Uncertain Capacitated Arc Routing Problem, Evolutionary Multi-Objective Optimisation

ACM Reference Format:
Shaolin Wang, Yi Mei, and Mengjie Zhang. 2021. Two-Stage Multi-Objective Genetic Programming with Archive for Uncertain Capacitated Arc Routing Problem. In *Proceedings of the Genetic and Evolutionary Computation Conference 2021 (GECCO '21).* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
GECCO '21, July 10–14, 2021, Lille, France
© 2021 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The Capacitated Arc Routing Problem (CARP) [14] is a classic combinatorial optimisation problem which has been proven to be NP-hard. CARP has a variety of real-world applications, such as waste collection [2] and winter gritting [17]. The main goal of CARP is to serve a set of *required edges (tasks)* in a graph using a fleet of vehicles with the minimum cost. There have been extensive studies for solving CARP [14, 45]. Most previous studies assume that the problem parameters, such as task demand and travel cost, are fixed and known in advance. However, in reality, the environment is usually uncertain and the problem parameters need to be considered as stochastic. For example, in waste collection, the amount of waste to be collected varies from one day to another, and we cannot know the exact amount of the waste until the waste collection finished.

The Uncertain Capacitated Arc Routing Problem (UCARP) [29] was proposed to reflect the reality better than CARP. In UCARP, the demands and travel costs are modeled as random variables. In addition to the NP-hardness inherited from CARP, *route failure* is an extra main challenge in UCARP. A route failure occurs when the actual demand of a task exceeds the remaining capacity of the vehicle. In this case, the vehicle has to go to the depot to refill its capacity, and return to complete the remaining part of the failed service. This can lead to a huge recourse cost.

Traditional solution optimisation approaches, such as mathematical programming [16] and evolutionary algorithms [22] are less effective when applied to UCARP because they cannot handle the routing failure effectively. Most traditional solution optimisation approaches try to preplan a robust solution. However, a single robust solution may not be flexible enough to cope with all the possible environments. On the other hand, re-optimising the remaining solution from scratch when unexpected events (e.g. route failure) occur may further improve the effectiveness, but the re-optimisation usually takes time and cannot react in real time.

The routing policy-based approaches [33] deal with the uncertain environment in UCARP by making routing decisions on-the-fly. In contrast to the traditional solution optimisation approaches, the routing policy-based approaches do not optimise a solution beforehand. Instead, they construct the routes based on the latest information in real time. Specifically, whenever a vehicle becomes idle (completes the current task), a routing policy is used to calculate the priority of each remaining task, and the next task for the vehicle is selected based on the priority. Some existing construction heuristics such as the path scanning heuristic [22] can also be seen as a routing policy. The routing policies can respond to the uncertain environment immediately. In addition, routing policies can provide quite different solutions in different environments, thus are much more flexible than the robustness optimisation approaches.

The effectiveness of routing policy depends on a variety of factors, such as the graph topology, the distributions of the random task demands and travel costs, and the objective(s) [20]. Therefore, manually designing an effective routing policy for a given problem is time-consuming, and relies highly on domain expertise. To address this issue, Genetic Programming Hyper-heuristic (GPHH) is a promising technique to automatically evolve effective routing policies. GPHH has been successfully applied to various dynamic problems, such as scheduling [9, 47, 49] and resource allocation [35, 36]. Recently, it has also been applied to UCARP [1, 24, 25, 27, 31].

Previous studies mainly focus on the effectiveness of the GP-evolved rules. However, the size (number of nodes) of the GP-evolved rules is also important, since smaller rules tend to have better generalisation and interpretability. Currently, the GP-evolved routing policies are typically very large as GP tends to continuously increase the size of its individuals, known as *bloat* [8, 10]. Therefore, it is necessary to optimise the size along with the effectiveness. The effectiveness and size are potentially conflicting objectives, thus it is natural to optimise them simultaneously using Multi-Objective GP (MOGP) [51], to obtain a set of routing policies with different trade-offs between these two objectives.

However, there are two main challenges for designing MOGP to obtain both effective and small routing policies for UCARP. First, during the evolutionary process, it is much easier for GP to generate small (but ineffective) routing policies than effective (and large) ones. If the two objectives of effectiveness and size are treated as equally important during the evolution, the population tends to be fully occupied by small but ineffective routing policies, and cannot cover the whole Pareto front. Second, the evaluation of the routing policy effectiveness is typically stochastic. Note that GPHH is essentially a learning approach rather than direct optimisation, and the ultimate goal is to apply the evolved routing policies to future unseen UCARP instances. Generally, obtaining accurate fitness values requires a large number of independent samples, but it is time consuming, and the use of a small number of samples may mislead GP searches. To improve the generalisation of the evolved routing policies, a common strategy is to use a small subset of training samples and change the samples at each generation (so-called *training sample rotation*). Such strategy has been widely used in other problems such as dynamic scheduling (e.g. [19, 46, 48, 50]) and is conceptually similar to the mini-batch strategy in machine learning. As a result, the effectiveness evaluation of a routing policies becomes stochastic and changes generation by generation. In this way, effective routing policies may be lost simply because its effectiveness happens to be not so promising in the small subset of training samples in a particular generation. This issue becomes more serious in MOGP in our problem, since it is already difficult to retain the potentially effective routing policies. Previous studies [8, 41, 43] mainly focused on the first issue, but ignored the second issue caused by the training sample rotation.

The overall goal of this paper is to develop new a MOGP algorithm for evolving effective and small routing policies for UCARP. To achieve this goal, we have the following research objectives:

(1) To address the bias towards the size during the GP process by a two-stage algorithm framework. The first stage forces the bias towards the effectiveness and the second stage treats the two objectives equally. This can reach a much better balance between the effectiveness and size.
(2) To address the issue caused by the stochastic effectiveness evaluation, we propose to use an external archive to store the potentially effective individuals that could have been lost during the traditional GP process. The individuals in the archive contribute back to the population in the breeding process.
(3) To develop a novel MOGP for UCARP combined with the two-stage framework, external archive and NSGAII [11]. The resultant algorithm is called the Two-Stage NSGPII with Archive (TSNSGPII-a)
(4) To verify the performance of the TSNSGPII-a on a set of UCARP instances, and analyse the evolved routing policies.

2 BACKGROUND

2.1 Problem Description

UCARP is based on a connected graph $G(V, E)$, where V is the set of vertices and E is the set of edges. Each edge $e \in E$ has a positive random deadheading cost $\tilde{c}_d(e)$, referring to the cost of traversing the edge. A subset of edges $E_t \subseteq E$ are required to be served. Each required edge (also called *task*) $e_t \in E_t$ has a positive random demand $\tilde{d}(e_t)$ and a positive serving cost $c_s(e_t)$ which refers to the cost of serving a task. A set of vehicles with capacity Q are allocated to serve the tasks in E_t. There is a depot $v_0 \in V$ in the graph. The goal is to minimise the total cost of serving all the tasks in E_t, subject to the following constraints. First, a vehicle must start and finish at v_0; Second, the total demand served between two subsequent depot visits of a vehicle cannot exceed the vehicle capacity; Third, each required edge is served exactly once.

A UCARP instance contains many random variables, each of which can have (an infinite number of) different sample values. Accordingly, a UCARP instance contains different samples. In a UCARP instance *sample*, each random variable, e.g. the task demand $\tilde{d}(e_t)$ or deadheading cost $\tilde{c}_d(e)$, takes a realised sample value. Note that a UCARP instance sample is similar to a traditional static CARP instance. However, in a UCARP instance sample, the actual demand of a task is unknown until it has been served, and the actual deadheading cost of an edge is unknown and is revealed when traversing over the edge. The random variables are assumed to follow the truncated normal distribution [24, 27, 31].

$$\tilde{d}(e_t) \sim \mathcal{N}(d(e_t), \frac{d(e_t)}{5}), \tilde{c}_d(e) \sim \mathcal{N}(c_d(e), \frac{c_d(e)}{5}). \quad (1)$$

Two kinds of failures may occur during the execution process because of the uncertain environment. The first one is the *route failure*. It occurs when the actual demand of the task being served is larger than expected, and exceeds the remaining capacity of the vehicle. The second one is the *edge failure*. It occurs when an edge in the planned route becomes inaccessible (e.g. due to unexpected roadworks). These failures can be handled by different strategies. Typically, the route failure is handled with a simple recourse operator that sends the vehicle to the depot to refill its capacity, and then return to the failed task to complete its remaining service. The edge

failure can be handled by a detour (e.g., use Dijkstra's algorithm to find a shortest path under the current situation).

2.2 Related Work

CARP is a classic combinatorial optimisation problem, which has been proven to be NP-hard [45]. Many approaches have been proposed to solve CARP. Golden and Wong [16] developed an integer linear programming model and solved CARP using the branch-and-cut algorithm. Belenguer and Benavent [6] applied a cutting plane algorithm to CARP. It is guaranteed that the obtained solutions are optimal by using these *exact* approaches. However, the approach is limited to only small-scale instances due to the NP-hardness of CARP. These approaches can become time consuming as the problem becomes large. To obtain sufficiently good solutions in a limited time budget and for larger problem instances, construction heuristics are also applied to CARP [16]. Construction heuristics generate approximated solutions from scratch. Although these approaches cannot guarantee the optimality of the obtained solutions, they are much less time consuming than the exact approaches. At the same time, they can usually provide good enough solutions within the given time budget. Path scanning heuristic, augment-merge heuristic, and Ulusoy's split heuristic are commonly used construction heuristics for CARP [22]. To make an improvement, meta-heuristic approaches are utilised in recent years. They are usually much less time consuming than the exact approaches. A variety of meta-heuristic algorithms, such as tabu search [18], genetic algorithm [21], memetic algorithms [22, 30, 37] and ant colony optimisation [13] have been proposed to CARP.

Traditional solution optimisation approaches are not directly applicable to UCARP, due to the route and edge failures caused by the uncertain environment. If not handled properly, the recourse can induce a large extra cost. Therefore, new approaches have been proposed for UCARP. So far, there are two main types of approaches for solving UCARP, the proactive approaches [15, 39, 40] and the reactive approaches [24, 28, 42].

The main idea of proactive approaches is to find robust solutions that can handle all the possible realisations of the random variables. Typically, the output of a proactive approach consists of two main components, a robust pre-planned solution and a recourse operator. Firstly, the optimisation algorithm produces a robust solution based on the prediction of the environment (distribution-based or sample-based). While executing the robust solution in practice, the recourse operators (e.g. refill and detour) are applied to adjust the solution on-the-fly when failures occur. The performance of the proactive approaches are highly affected by the accuracy of the prediction of the stochastic environment and the effectiveness of the recourse operators.

Reactive approaches mainly focus on designing effective routing policies that can make real-time decisions during the execution process, and construct the solution on-the-fly. Weise et al. [44] first proposed a GPHH to evolve routing policies for Stochastic CARP (SCARP), and examined its performance on a single-vehicle simulation. However, SCARP only consider the random presence of tasks but ignore the stochastic demand. Mei et al. [29] proposed UCARP which consider four different stochastic factors. Liu et al. [24] proposed a filter mechanism which can filter irrelevant candidate tasks before applying the routing policy during the decision-making process. Then, Mei et al. [31] extended the model from single-vehicle to multiple-vehicle version. MacLachlan et al. [27] proposed a novel task filtering method and an effective look-ahead terminal to improve the GPHH further, and proposed a more effective recourse operator with closer collaborations among vehicles [28]. Recently, Ardeh et al. [3–5] utilised transfer learning to speed up the training process in new scenarios. Liu et al. [25] proposed a new predictive-reactive approach with Genetic Programming, cooperative coevolution was applied to evolve task sequence and a recourse policy simultaneously.

Previous studies have shown the success of GPHH on evolving effective routing policies. However, the GP-evolved routing policies are typically very large and complex. One main reason is that the individuals in GP tend to grow too large unnecessarily during the evolutionary process, which is called *bloat*. Thus, it is important to control the tree size while ensuring the effectiveness. Tree size/depth limitation is the most straightforward method to control tree sizes [32]. However, it is difficult to set a proper limit for size/depth without extensive trial and error [8]. A penalty term can be utilised to control the tree size as well, also called "parsimony pressure". The main idea of the penalty term is to penalize the larger trees and reward the smaller trees. The penalty term suffers from a similar limitation with the tree size/depth limitation. It is hard to set a proper weight of the penalty term without extensive trial and error [10, 34]. Evolutionary Multi-Objective Optimisation (EMO) approaches can be also utilised to control the tree size [8]. The EMO approaches do not have to predefine any weights or tree size limitation. They can find a set of solutions with different trade-offs between the effectiveness and size. Several famous EMO approaches include NSGAII [11], SPEA2 [8] and MOEA/D [52].

When optimising effectiveness and size simultaneously by Multi-Objective GP (MOGP), one issue is that the size is much easier to optimise than effectiveness. In other words, GP can generate small individuals much more easier than effective individuals. If using the traditional EMO approaches directly, the population tends to bias to the small (but ineffective) individuals. This is the so-called objective selection bias issue [10, 23]. Bleuler [7] utilised the SPEA2 to deal with this issue. The proposed algorithm intentionally increases the diversity of the population, and force the search to bias against the size to some extent. While preference-based EMO approaches [12, 38] may also be considered to control the bias between the effectiveness and size, they typically focus only on a preferred region of the Pareto front, while in our problem we still aim to cover the entire Pareto front so that users can make their selection from the comprehensive Pareto front. Therefore, the preference-based EMO approaches are not directly applicable here.

3 THE PROPOSED APPROACH

3.1 Overall Framework

The flowchart of the proposed TSNSGPII-a is given in Fig. 1. It consists of two stages. Both stages utilise an external archive to store potentially good individuals. The first stage is from generation 0 to generation $G/2$ (G is the maximal number of generations), i.e. half of the entire search process. During the first stage, we only consider

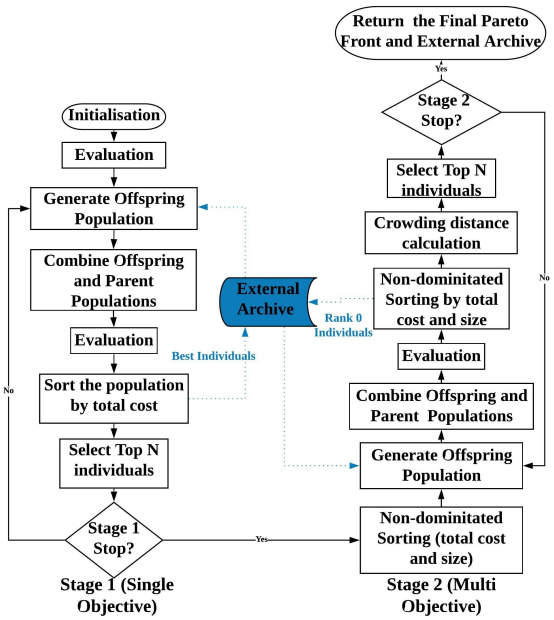


Figure 1: Overview of TSNSGPII-a.

minimising the total cost, and ignore the size temporarily. In other words, the first stage is a single-objective optimisation stage. Initially, the population is randomly generated and an empty external archive is created. Then, each individual is evaluated purely based on the effectiveness (the calculation will be given by Eq. (2)). At each generation, the offspring population is first generated from the current population and the external archive by the evolutionary operators (e.g. crossover and mutation). After that, the current and offspring populations are combined and evaluated. Finally, the individuals in the combined population are sorted based on the total cost, and the top N (N is the population size) individuals are selected to be in the next generation. Note that the first stage of TSNSGPII-a is different from the standard single-objective GP, which directly passes all the offspring to the next generation. Therefore, the first stage of TSNSGPII-a is greedier than the standard GP. In addition, due to the training sample rotation, the parents are re-evaluated on the new training samples to be compared with the offspring in the combined population. The second stage of TSNSGPII-a is from generation $G/2+1$ to generation G. This stage considers both the objectives of total cost and size, and treats them as equally important. It follows the NSGA-II [11] framework, except that at each generation the parents are selected from both the current population and the external archive. At each generation, the offspring population is first generated from the current population and the archive. Then, the current offspring populations are combined with the parent population, and non-dominated sorting and crowding distance calculation are applied. Finally, the top N individuals are selected into the next generation. At each generation, the external archive is updated with the non-dominated individuals in the population, and contributes itself to generate offspring during the breeding process. The details can be seen in Section 3.4.

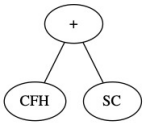


Figure 2: An example of a simple routing policy.

3.2 Individual Representation

In TSNSGPII-a, a routing policy is essentially an arithmetic priority function. Given a decision situation (i.e. a vehicle completes its current task, and is to decide the next task to serve), the routing policy calculates the priority value of each remaining task and selects the most prioritized task for the vehicle to serve next. In GP, a priority function can be represented as a tree. Fig. 2 shows an example of a routing policy, represented as "CFH + SC", where CFH indicates the travel cost from the candidate task to current location, and SC is the serving cost of the candidate task. This routing policy prefers the tasks with lower serving cost and closer to current location.

3.3 Fitness Evaluation

Given a routing policy rp, the effectiveness $eff(rp)$ is defined as the average quality (i.e. total cost) of the solution constructed by applying rp to a set of instance samples $\mathcal{E}$. Specifically,

$$f_1(rp) = eff(rp) = \frac{1}{|\mathcal{E}|}\sum_{e\in\mathcal{E}} tc(rp, e), \tag{2}$$

where $tc(rp, e)$ refers to the total cost of the solution obtained by rp on sample e and $|\mathcal{E}|$ stands for the number of instance samples. The size of rp is simply calculated by counting the nodes in rp:

$$f_2(rp) = \text{number of nodes in } rp \tag{3}$$

For example, the routing policy in Fig. 2 has a size of 3. These two objectives are used in the non-dominated sorting and crowding distance calculation.

3.4 External Archive

The external archive is used to store the potentially good individuals generated during the evolutionary process. As shown in Fig. 1, potentially good individuals are put into the external archive at the end of each generation. The external archive contains genotypically unique individuals. Whenever a new individual is to be added into the archive, its genotype (i.e. tree structure) is compared with that of the individuals in the archive. If there is a duplicate already in the archive, the new individual will be discarded. The maximal size of external archive is set the same as the population size, i.e. the archive can only contain as many individuals as the population. When the size of the archive is exceeded, individuals will be re-evaluated, then non-dominated sorting is applied to the individuals in the archive, and only retain population size non-dominated individuals. The pseudocode of the updating process of the external archive is shown in Algorithm 1.

The external archive contributes itself during the breeding process. Specifically, when selecting the parents for crossover and mutation, the current population is combined with the external archive to form the mating pool. Then, the parents are selected

Algorithm 1: The updating process of the external archive.

Input: The current number of generation g, The total number of generation G, the current population pop, the external $archive$, the population size $popsize$
Output: The updated $archive$

```
if g ≤ G/2 then
    // Stage 1
    Obtain the best individual in the population:
      ind = arg min_{i∈pop} f_1(i) (Eq. (2));
    Set duplicate = false;
    for ind' ∈ archive do
        if genotype(ind) equals to genotype(ind') then
            duplicate = true;
            break;
        end
    end
    if duplicate = false then
        Re-assign the rank of ind to 0.5;
        Put ind into archive;
    end
else
    // Stage 2
    Obtain the non-dominated (rank-0) solutions in pop:
      pop_nd = {i ∈ pop | rank(i) = 0};
    for ind ∈ pop_nd do
        Set duplicate = false;
        for ind' ∈ archive do
            if genotype(ind) equals to genotype(ind') then
                duplicate = true;
                break;
            end
        end
        if duplicate = false then
            Re-assign the rank of ind to 0.5;
            Put ind into archive;
        end
    end
end
if |archive| ≥ popsize then
    Non-dominated sort the archive;
    Remove duplicates;
    Retain popsize individuals archive_nd;
    Set archive = archive_nd;
end
return archive;
```

from the combined mating pool by the tournament selection (of size 7). To avoid re-applying the non-dominated sorting and crowding distance calculation on the combined mating pool, we define the following archive-aided comparison.

- Set all the individuals in the archive as rank 0.5;
- Individual A is better than individual B if $rank(A) < rank(B)$;
- If $rank(A) = rank(B)$, then
  - if A and B are from the current population, then we use the pre-calculated crowding distance, and select the one with a larger crowding distance.
  - if A and B are from the archive, then we select them randomly.

The pseudocode of the archive-aided breeding process is shown in Algorithm 2.

Algorithm 2: The archive-aided breeding process.

Input: The current population pop, the external $archive$
Output: The offspring population pop'

```
Create an empty population pop' = ∅;
Set the mating pool as pool = archive ∪ pop;
while |pop'| < popsize do
    Select the parents by the tournament selection with
      the archive-aided comparison;
    Generate a child using parents by crossover/mutation;
    Add child to pop';
end
return pop';
```

4 EXPERIMENTAL STUDIES

To verify the goodness of the proposed TSNSGPII-a, we compare it with several state-of-the-art MOGPs for UCARP. Specifically, we compare with the SPEA2 [8] that weakens the bias towards size, the α-MOGP [43] which is the current state-of-the-art MOGP approach for UCARP in terms of effectiveness and size, NSGPII without the two-stage framework and archive and TSNSGPII with the two-stage framework but no archive. We also compare with the baseline single-objective GPHH in terms of effectiveness.

4.1 Experiment Setup

We select 6 representative UCARP instances from the commonly used Ugdb and Uval datasets [24, 27, 31] for experiments. The problem size of these instances varies from small (22 tasks and 5 vehicles) to large (97 tasks and 10 vehicles). The experiment is divided into the training and test phases. In the training phase, 5 training samples are used for evaluation, which is changed at each generation. In the test phase, the best trained routing policy is tested on an unseen test set containing 500 test samples.

The terminal set is shown in Table 1. The function set is set to $\{+, -, \times, /, \min, \max\}$ where "/" operator is protected division which returns 1 if divided by 0. The number of generations is 50 for all compared algorithms. For the single-objective GPHH, the population size is 1000. Since the NSGPII double the number of evaluations per generation due to the re-evaluations, for fair comparison, we set the population size to 500 for all the NSGPII algorithms. For other MOGPs, the population size is 1000. Ramp-half-and-half initialisation is used. The tournament selection size is 7. The maximal depth is 8. The crossover and mutation rates are 0.85 and 0.15. All the algorithms are implemented on Evolutionary Computation Java (ECJ) package [26]. Each algorithm was run 30 times independently for each UCARP instance.

Table 1: The terminal set in the experiments.

Terminal	Description
CR	cost from the current location to the depot
DEM	expected demand of the candidate task
DEM1	demand of unserved task that closest to the candidate task
SC	cost of serving the candidate task
CFD	cost from the head node of the task to the depot
CFH	cost from the candidate task to the current location
FULL	fullness (served demand over capacity) of the vehicle
FUT	fraction of unassigned tasks
FRT	fraction of unserved tasks
CTD	cost from the depot to the candidate task
RQ	remaining capacity of the vehicle
RQ1	remaining capacity of the closest other route to the candidate task
CTT1	cost from the candidate to its closest remaining task
CFR1	cost from the closest other route to the candidate task
ERC	a random constant value

4.2 Results and Discussions

4.2.1 Comparison with State-of-the-art approach. Tables 2 and 3 shows the results of the Hyper-Volume (HV) and Inverted Generational Distance (IGD) of the SPEA2, α-MOGP and TSNSGPII-a on the 6 test instances. For HV, we normalised the objective values first, and then use (1,1) as reference point. For IGD, we exact all the solutions from all algorithms for each instance and find a reference Pareto front from these solutions. For each instance, the Wilcoxon rank sum test with a significance level of 0.05 is conducted between the 30 independent runs of TSNSGPII-a and each compared algorithm. If TSNSGPII-a is statistically significantly better than, worse than, and comparable with that of the compared algorithm, the corresponding entry is marked with (+), (-), or (=), respectively.

From the tables, we can see that TSNSGPII-a perform best among the compared algorithms. It can outperform SPEA2 and α-MOGP on all the instances in both HV and IGD. Fig. 3 plots the non-dominated set of single run that has the median HV value of the 30 runs of each compared algorithm on each instance. It can be seen that the solutions obtained by TSNSGPII-a are better distributed in the objective space and can mostly dominate that obtained by SPEA2 and α-MOGP. It can be seen that the solutions of SPEA2 are biased to size, and cannot achieve good effectiveness. Typically, SPEA2 can maintain the diversity of the population as it has an internal archive to store good individuals. However, in our problem, we do training sample rotation every generation. In this case, the internal archive of SPEA2 is not able to maintain the diversity of the population. We can see that α-MOGP is sensitive to instance. It can obtain good solutions on 4 out 6 instances. However, it is still slightly biased to size on Ugdb2 and Ugdb23. Overall, the results indicate that TSNSGPII-a can generate better non-dominated solutions than the state-of-the-art MOGP approaches, and it is much easier for users to choose a routing policy from the Pareto front from TSNSGPII-a based on their preference.

4.2.2 Effectiveness of the External Archive. The use of the external archive is a major contribution in TSNSGPII-a. To verify the effectiveness of the external archive, we compare TSNSGPII-a with TSNSGPII, which is the version without the archive. In addition, we also compare NSGPII-a with NSGPII. Both the algorithms have a single stage, which is the second stage of TSNSGPII-a. NSGPII-a has the archive, while NSGPII has not.

Table 2: The mean and standard deviation for HV of the compared algorithms in test process.

Instance	SPEA2	α-MOGP	TSNSGPII-a
Ugdb1	0.8676(0.04)	0.9446(0.03)	0.9643(0.01)(+)(+)
Ugdb2	0.8953(0.04)	0.9524(0.03)	0.9674(0.01)(+)(+)
Ugdb8	0.8378(0.05)	0.8999(0.06)	0.9673(0.01)(+)(+)
Ugdb23	0.8664(0.03)	0.9366(0.03)	0.9509(0.02)(+)(+)
Uval9D	0.8524(0.05)	0.9326(0.04)	0.963(0.01)(+)(+)
Uval10D	0.9019(0.03)	0.9553(0.04)	0.9779(0.01)(+)(+)

Table 3: The mean and standard deviation for IGD of the compared algorithms in test process.

Instance	SPEA2	α-MOGP	TSNSGPII-a
Ugdb1	0.1022(0.04)	0.058(0.02)	0.0381(0.01)(+)(+)
Ugdb2	0.0814(0.03)	0.0544(0.01)	0.0325(0.01)(+)(+)
Ugdb8	0.2637(0.04)	0.2002(0.03)	0.0799(0.03)(+)(+)
Ugdb23	0.1124(0.03)	0.0873(0.02)	0.0412(0.01)(+)(+)
Uval9D	0.2104(0.05)	0.1262(0.03)	0.0604(0.03)(+)(+)
Uval10D	0.089(0.03)	0.0496(0.01)	0.0268(0.01)(+)(+)

Tables 4 and 5 show the results of the Hyper-Volume (HV) and Inverted Generational Distance (IGD) of NSGPII, NSGPII-a, TSNSGPII and TSNSGPII-a. We can see that NSGPII-a can outperform NSGPII in HV for 5 out of 6 instances, and in IGD for all instances. In addition, it can be seen that TSNSGPII-a can outperform TSNSGPII in both HV and IGD for all the 6 instances. Overall, we can see that the external archive can significantly improve the performance of the MOGP, no matter whether the two-stage framework is used or not.

Table 4: The HV of NSGPII, NSGPII-a, TSNSGPII and TSNSGPII-a.

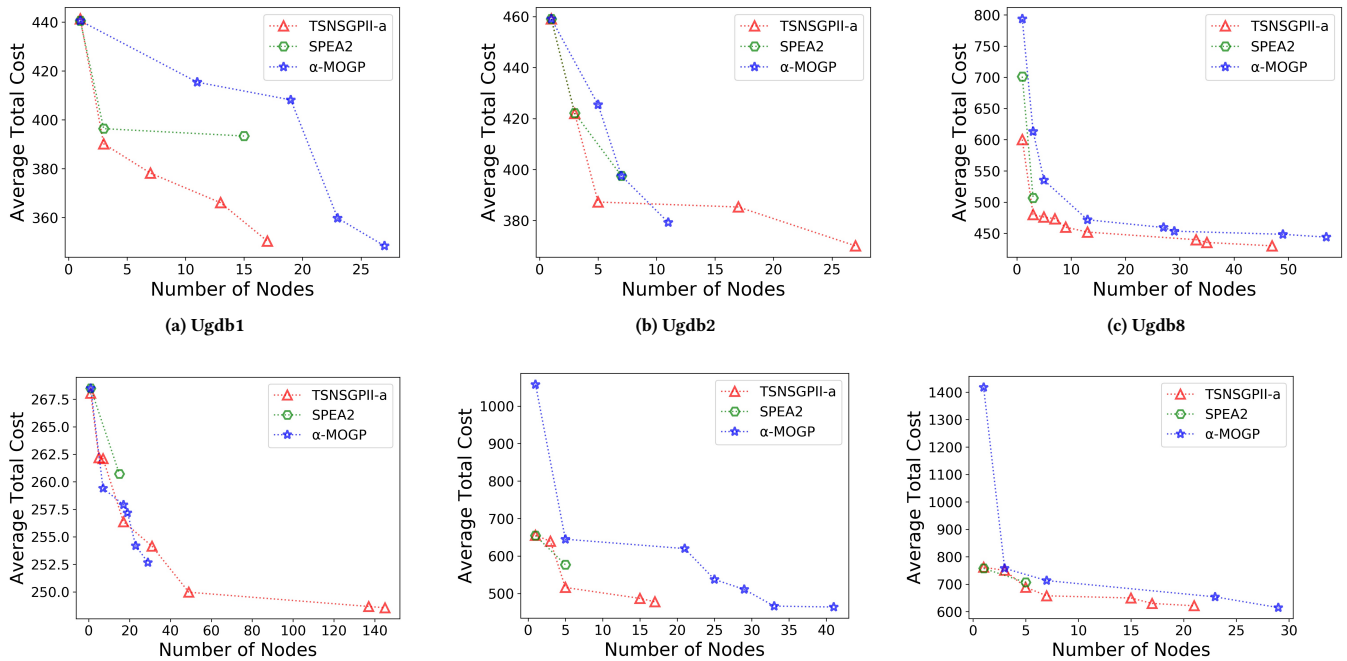
Instance	NSGPII	NSGPII-a	TSNSGPII	TSNSGPII-a
Ugdb1	0.9104(0.03)	0.9333(0.02)(+)	0.9272(0.02)	0.9643(0.01)(+)
Ugdb2	0.9217(0.02)	0.9355(0.02)(+)	0.9383(0.02)	0.9674(0.01)(+)
Ugdb8	0.897(0.03)	0.9544(0.01)(+)	0.9046(0.03)	0.9673(0.01)(+)
Ugdb23	0.8826(0.02)	0.9135(0.02)(+)	0.8964(0.03)	0.9509(0.02)(+)
Uval9D	0.9184(0.02)	0.9227(0.02)(=)	0.9389(0.01)	0.963(0.01)(+)
Uval10D	0.9336(0.02)	0.9452(0.02)(+)	0.9572(0.02)	0.9779(0.01)(+)

Table 5: The IGD of NSGPII, NSGPII-a, TSNSGPII and TSNSGPII-a.

Instance	NSGPII	NSGPII-a	TSNSGPII	TSNSGPII-a
Ugdb1	0.0669(0.02)	0.056(0.01)(+)	0.058(0.02)	0.0381(0.01)(+)
Ugdb2	0.0591(0.01)	0.0531(0.01)(+)	0.0544(0.01)	0.0325(0.01)(+)
Ugdb8	0.2144(0.03)	0.1243(0.03)(+)	0.2002(0.03)	0.0799(0.03)(+)
Ugdb23	0.0981(0.02)	0.0689(0.02)(+)	0.0873(0.02)	0.0412(0.01)(+)
Uval9D	0.1563(0.02)	0.116(0.03)(+)	0.1262(0.03)	0.0604(0.03)(+)
Uval10D	0.0648(0.02)	0.0535(0.02)(+)	0.0496(0.01)	0.0268(0.01)(+)

4.3 Further Analysis

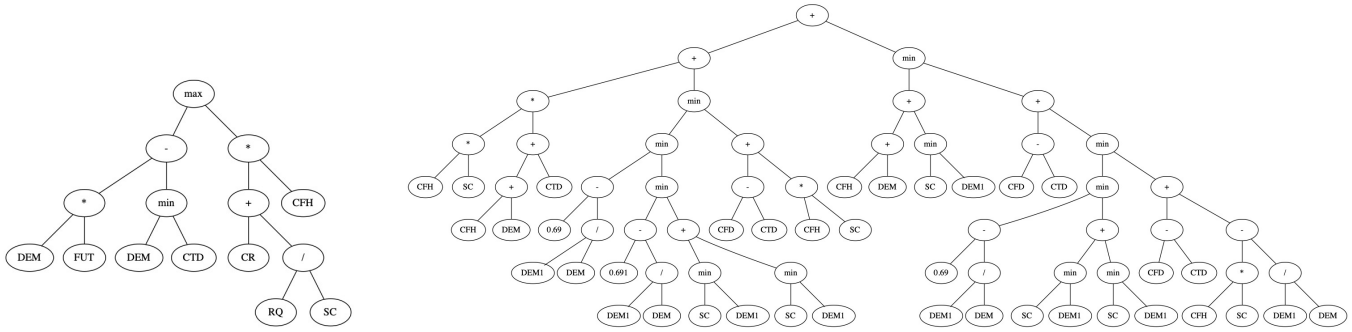
The ultimate goal of GPHH for UCARP is to obtain an effective and small routing policy. Here, we assume that the effectiveness is preferred, and the routing policy from the final set with the lowest total cost is selected. Then, we compare the selected routing policy from those obtained by TSNSGPII-a with the routing policy obtained by the baseline single-objective GPHH [24].



(a) Ugdb1 (b) Ugdb2 (c) Ugdb8

(d) Ugdb23 (e) Uval9D (f) Uval10D

Figure 3: The non-dominated solution of the run with the median HV among the 30 runs of each compared algorithm.



(a) TSNSGPII-a (b) GPHH

Figure 4: Example routing policies obtained by TSNSGPII-a and GPHH.

Table 6 shows the mean and standard deviation of the average total cost (effectiveness) of TSNSGPII-a and GPHH, along with that of SPEA2 and α-MOGP. We can see that TSNSGPII-a outperformed GPHH on 2 out of 6 instances, and achieved comparable effectiveness on the other instances. Besides, TSNSGPII-a can outperform SPEA2 on all instances. This is consistent with the patterns shown in Fig. 3. The median non-dominated set of SPEA2 is biased to size. TSNSGPII-a also achieved comparable results with α-MOGP on 5 out 6 instances and outperform α-MOGP on 1 instance.

Table 7 shows the mean and standard deviation of the average number of nodes (size) of compared algorithms. We can see that SPEA2 achieves the smallest size. This is because its search process is still biased to the size, and the effectiveness is largely ignored due to the objective selection bias issue. TSNSGPII-a obtains significantly smaller routing policies than α-MOGP on 3 out of 6 instances. This is consistent with Fig. 3. TSNSGPII-a can obtain comparable effectiveness with α-MOGP but with smaller size. Therefore, the median non-dominated set of TSNSGPII-a can dominate that of α-MOGP. Overall, TSNSGPII-a can evolve significantly smaller routing policies than GPHH without sacrificing effectiveness on all instances. This verifies the effectiveness of the two-stage framework and external archive used in TSNSGPII-a.

4.3.1 Semantic Analysis. We randomly select two best routing policies from GPHH and TSNSGPII-a for Ugdb23, one for each

Table 6: The mean and standard deviation of the average total cost of best individual from each compared algorithms.

Instance	GPHH	SPEA2	α-MOGP	TSNSGPII-a
Ugdb1	355.47(14.83)	366.54(9.04)	354.77(12.05)	350.55(5.34)(=)(+)(=)
Ugdb2	371.72(7.67)	384.02(11.53)	370.17(7.91)	370.18(5.89)(=)(+)(=)
Ugdb8	443.34(54.3)	468.61(19.12)	441.05(10.75)	433.17(8.48)(+)(+)(+)
Ugdb23	252.47(3.11)	258.23(4.11)	250.47(2.16)	250.91(2.3)(+)(+)(=)
Uval9D	478.14(16.68)	500.72(14.71)	474.24(11.94)	475.01(7.9)(=)(+)(=)
Uval10D	620.91(7.97)	644.7(18.95)	619.59(8.45)	621.63(7.54)(=)(+)(=)

Table 7: The mean and standard deviation of the size of the best individual from each compared algorithms.

Instance	GPHH	SPEA2	α-MOGP	TSNSGPII-a
Ugdb1	74.6(23.84)	12.13(9.41)	27.67(15.97)	46.33(34.29)(+)(-)(-)
Ugdb2	71.93(23.79)	16.27(12.83)	28.8(13.88)	36.67(26.14)(+)(-)(=)
Ugdb8	65.47(24.33)	16.07(20.08)	51.67(19.03)	45.47(24.12)(+)(-)(=)
Ugdb23	71.8(25.22)	14.4(13.93)	47.53(26.57)	35.53(28.19)(+)(-)(+)
Uval9D	69.27(29.46)	20.3(15.16)	58.0(29.24)	40.47(17.24)(+)(-)(+)
Uval10D	65.33(14.35)	11.4(5.39)	47.67(23.7)	34.67(17.95)(+)(-)(+)

algorithm. These two routing policies are shown in Fig. 4. The effectiveness of the routing policy obtained by GPHH is 249.89, and the effectiveness of the routing policy obtained by TSNSGPII-a is 249.27. It can be seen that the routing policy from GPHH is large and complex. We cannot gain useful knowledge from it easily. In contrast, it is easier to understand the routing policy from TSNSGPII-a as it is smaller and simpler. The routing policy obtained by TSNSGPII-a can be represented as follows:

$$RP = \max(S_1, S_2)$$

where

$$S_1 = FUT * DEM - \min(DEM, CTD),$$
$$S_2 = (CR + \frac{RQ}{SC}) * CFH$$

To gain detailed understandings of the behaviours from the routing policies evolved by TSNSGPII-a and make it easier to understand, we transform RP to the following IF-ELSE format rule set.

if $S_1 \geq S_2$ **then**
  if $DEM \leq CTD$ **then**
    $RP = FUT * DEM - DEM$
  else
    $RP = FUT * DEM - CTD$
  end if
else
  $RP = (CR + \frac{RQ}{SC}) * CFH$
end if

We can extract the following patterns and understandings from the rule set:

- When $S_1 \geq S_2$, RP becomes S_1. There are two possible cases for $S_1 \geq S_2$ to happen:
  - S_1 is large. This indicates that there are many remaining tasks, and all the remaining tasks have large demand and are close to the depot.
  - S_2 is small. This indicates that vehicle is relatively empty and is close to the depot. All the remaining tasks are close to the current location.

  There are two possible cases of S_1:
  - When $DEM \leq CTD$, S_1 becomes $FUT * DEM - DEM$ or $(FUT - 1) * DEM$. Note that, FUT refers to the fraction of the unassigned tasks. The value of FUT is between 0 and 1. It is hard for $(FUT - 1)$ to greater than 0. Thus, the output of $(FUT - 1) * DEM$ will be a negative value. This leads to a negative output of S_1. However, this contradicts the assumption that $S_1 \geq S_2$ because the output of S_2 is usually a positive value. Therefore, this case would never happen.
  - When $DEM > CTD$, which means that all the remaining tasks are very close to the depot and require large demand. RP becomes $FUT * DEM - CTD$.

  In this case, RP prefers the tasks with small demands and far away from the depot.
- Otherwise, $S_1 < S_2$ indicates that the vehicle is far away from the depot, and there are only a few remaining tasks with small demands, all the remaining tasks are close to the current location of the vehicle and far away from the depot. RP becomes $(CR + \frac{RQ}{SC}) * CFH$, and RP prefers the tasks with large serving cost and close to the current location.

5 CONCLUSIONS AND FUTURE WORK

This paper aims to develop MOGP that can evolve both effective and smaller routing policies for UCARP. This goal has been achieved by proposing a new Two-Stage Non-dominated Sorting GP-II with external archive (TSNSGPII-a). TSNSGPII-a contains a two-stage framework that explicitly controls the balance between the effectiveness and size during the evolutionary process. The external archive stores a wide range of potentially effective individuals, in case they are lost from the population due to the training sample rotation. The experimental results showed that the proposed TSNSGPII-a managed to obtain both effective and smaller routing policies than the state-of-the-art MOGPs for UCARP. It also provides a better distributed set of routing policies, making it easier for the end-users to select better routing policy based on their preferences. We have also verified the efficacy of the external archive used in TSNSGPII-a. The external archive can significantly improve the performance of the MOGP in terms of HV and IGD. The routing policies evolved by TSNSGPII-a is much easier to understand than that in GPHH. In the future, we will try to propose more effective way to update the external archive, try to find promising way to reevaluate the individual in the external archive to ensure the external archive can only contain good individuals, and combine the external archive with other MOGP algorithms, and develop new approach that can further reduce the size and improve the effectiveness.

REFERENCES

[1] Harith Al-Sahaf, Ying Bi, Qi Chen, Andrew Lensen, Yi Mei, Yanan Sun, Binh Tran, Bing Xue, and Mengjie Zhang. 2019. A survey on evolutionary machine learning. *J. Roy. Soc. New Zeal.* 49, 2 (2019), 205–228.
[2] S.K. Amponsah and S. Salhi. 2004. The Investigation of a Class of Capacitated Arc Routing Problems: The Collection of Garbage in Developing Countries. *Waste Management* 24, 7 (2004), 711–721.
[3] Mazhar Ansari Ardeh, Yi Mei, and Mengjie Zhang. 2019. Genetic programming hyper-heuristic with knowledge transfer for uncertain capacitated arc routing problem. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 334–335.
[4] Mazhar Ansari Ardeh, Yi Mei, and Mengjie Zhang. 2020. Diversity-driven Knowledge Transfer for GPHH to Solve Uncertain Capacitated Arc Routing Problem. In *Proceedings of the IEEE Symposium Series on Computational Intelligence*. IEEE, 2407–2414.
[5] Mazhar Ansari Ardeh, Yi Mei, and Mengjie Zhang. 2020. Genetic Programming Hyper-Heuristics with Probabilistic Prototype Tree Knowledge Transfer for

Uncertain Capacitated Arc Routing Problems. In *Proceedings of the IEEE Congress on Evolutionary Computation*. IEEE, 1–8.

[6] José M Belenguer and Enrique Benavent. 2003. A cutting plane algorithm for the capacitated arc routing problem. *Computers & Operations Research* 30, 5 (2003), 705–728.

[7] Stefan Bleuler, Johannes Bader, and Eckart Zitzler. 2008. Reducing bloat in GP with multiple objectives. In *Multiobjective Problem Solving from Nature*. Springer, 177–200.

[8] Stefan Bleuler, Martin Brack, Lothar Thiele, and Eckart Zitzler. 2001. Multiobjective genetic programming: Reducing bloat using SPEA2. In *Proceedings of the Congress on Evolutionary Computation*. IEEE, 536–543.

[9] Juergen Branke, Su Nguyen, Christoph W Pickardt, and Mengjie Zhang. 2015. Automated design of production scheduling heuristics: A review. *IEEE Transactions on Evolutionary Computation* 20, 1 (2015), 110–124.

[10] Edwin D De Jong, Richard A Watson, and Jordan B Pollack. 2001. Reducing bloat and promoting diversity using multi-objective methods. In *Proceedings of the Conference on Genetic and Evolutionary Computation*. 11–18.

[11] Kalyanmoy Deb, Samir Agrawal, Amrit Pratap, and Tanaka Meyarivan. 2000. A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. In *Proceedings of the International Conference on Parallel Problem Solving from Nature*. Springer, 849–858.

[12] Kalyanmoy Deb and J Sundar. 2006. Reference point based multi-objective optimization using evolutionary algorithms. In *Proceedings of the Conference on Genetic and evolutionary computation*. ACM, 635–642.

[13] Karl F Doerner, Richard F Hartl, Vittorio Maniezzo, and Marc Reimann. 2004. Applying ant colony optimization to the capacitated arc routing problem. In *International Workshop on Ant Colony Optimization and Swarm Intelligence*. Springer, 420–421.

[14] Moshe Dror. 2012. *Arc routing: theory, solutions and applications*. Springer Science & Business Media.

[15] Gérard Fleury, Philippe Lacomme, and Christian Prins. 2004. Evolutionary algorithms for stochastic arc routing problems. In *Workshops on Applications of Evolutionary Computation*. Springer, 501–512.

[16] Bruce L Golden and Richard T Wong. 1981. Capacitated arc routing problems. *Networks* 11, 3 (1981), 305–315.

[17] H. Handa, L. Chapman, and Xin Yao. 2006. Robust route optimization for gritting/salting trucks: a CERCIA experience. *IEEE Computational Intelligence Magazine* 1, 1 (2006), 6–9.

[18] Alain Hertz, Gilbert Laporte, and Michel Mittaz. 2000. A tabu search heuristic for the capacitated arc routing problem. *Operations research* 48, 1 (2000), 129–135.

[19] T. Hildebrandt, J. Heger, and B. Scholz-Reiter. 2010. Towards improved dispatching rules for complex shop floor scenarios: a genetic programming approach. In *Proceedings of Genetic and Evolutionary Computation Conference*. ACM, 257–264.

[20] Josiah Jacobsen-Grocott, Yi Mei, Gang Chen, and Mengjie Zhang. 2017. Evolving heuristics for dynamic vehicle routing with time windows using genetic programming. In *IEEE Congress on Evolutionary Computation*. IEEE, 1948–1955.

[21] Philippe Lacomme, Christian Prins, and Wahiba Ramdane-Chérif. 2001. A genetic algorithm for the capacitated arc routing problem and its extensions. In *Workshops on Applications of Evolutionary Computation*. Springer, 473–483.

[22] Philippe Lacomme, Christian Prins, and Wahiba Ramdane-Cherif. 2004. Competitive memetic algorithms for arc routing problems. *Annals of Operations Research* 131, 1-4 (2004), 159–185.

[23] William B Langdon and JP Nordin. 2000. Seeding genetic programming populations. In *Proceedings of the European Conference on Genetic Programming*. Springer, 304–315.

[24] Yuxin Liu, Yi Mei, Mengjie Zhang, and Zili Zhang. 2017. Automated heuristic design using genetic programming hyper-heuristic for uncertain capacitated arc routing problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM, 290–297.

[25] Yuxin Liu, Yi Mei, Mengjie Zhang, and Zili Zhang. 2020. A predictive-Reactive approach with genetic programming and cooperative coevolution for the uncertain capacitated arc routing problem. *Evolutionary computation* 28, 2 (2020), 289–316.

[26] Sean Luke, Liviu Panait, Gabriel Balan, Sean Paus, Zbigniew Skolicki, Jeff Bassett, Robert Hubley, and A Chircop. 2006. Ecj: A java-based evolutionary computation research system. (2006). http://cs.gmu.edu/eclab/projects/ecj

[27] Jordan MacLachlan, Yi Mei, Juergen Branke, and Mengjie Zhang. 2018. An Improved Genetic Programming Hyper-Heuristic for the Uncertain Capacitated Arc Routing Problem. In *Proceedings of the Australasian Joint Conference on Artificial Intelligence*. Springer, 432–444.

[28] Jordan MacLachlan, Yi Mei, Juergen Branke, and Mengjie Zhang. 2019. Genetic Programming Hyper-Heuristics with Vehicle Collaboration for Uncertain Capacitated Arc Routing Problems. *Evolutionary computation* (2019), 1–29.

[29] Yi Mei, Ke Tang, and Xin Yao. 2010. Capacitated arc routing problem in uncertain environments. In *Proceedings of the IEEE Congress on Evolutionary Computation*. IEEE, 1–8.

[30] Yi Mei, Ke Tang, and Xin Yao. 2011. Decomposition-based memetic algorithm for multiobjective capacitated arc routing problem. *IEEE Transactions on Evolutionary Computation* 15, 2 (2011), 151–165.

[31] Yi Mei and Mengjie Zhang. 2018. Genetic Programming Hyper-heuristic for Multi-vehicle Uncertain Capacitated Arc Routing Problem. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. ACM, New York, NY, USA, 141–142.

[32] Riccardo Poli, William B Langdon, Nicholas F McPhee, and John R Koza. 2008. *A field guide to genetic programming*. Lulu. com.

[33] Ulrike Ritzinger, Jakob Puchinger, and Richard F Hartl. 2016. A survey on dynamic and stochastic vehicle routing problems. *International Journal of Production Research* 54, 1 (2016), 215–231.

[34] Terence Soule and James A Foster. 1998. Effects of code growth and parsimony pressure on populations in genetic programming. *Evolutionary computation* 6, 4 (1998), 293–309.

[35] Boxiong Tan, Hui Ma, and Yi Mei. 2018. A genetic programming hyper-heuristic approach for online resource allocation in container-based clouds. In *Proceedings of the Australasian Joint Conference on Artificial Intelligence*. Springer, 146–152.

[36] Boxiong Tan, Hui Ma, and Yi Mei. 2019. A hybrid genetic programming hyper-heuristic approach for online two-level resource allocation in container-based clouds. In *Proceedings of the IEEE Congress on Evolutionary Computation*. IEEE, 2681–2688.

[37] Ke Tang, Yi Mei, and Xin Yao. 2009. Memetic algorithm with extended neighborhood search for capacitated arc routing problems. *IEEE Transactions on Evolutionary Computation* 13, 5 (2009), 1151–1166.

[38] Lothar Thiele, Kaisa Miettinen, Pekka J Korhonen, and Julian Molina. 2009. A preference-based evolutionary algorithm for multi-objective optimization. *Evolutionary Computation* 17, 3 (2009), 411–436.

[39] Juan Wang, Ke Tang, Jose A Lozano, and Xin Yao. 2016. Estimation of the distribution algorithm with a stochastic local search for uncertain capacitated arc routing problems. *IEEE Transactions on Evolutionary Computation* 20, 1 (2016), 96–109.

[40] Juan Wang, Ke Tang, and Xin Yao. 2013. A memetic algorithm for uncertain capacitated arc routing problems. In *IEEE Workshop on Memetic Computing*. IEEE, 72–79.

[41] Shaolin Wang, Yi Mei, John Park, and Mengjie Zhang. 2019. A Two-Stage Genetic Programming Hyper-Heuristic for Uncertain Capacitated Arc Routing Problem. In *Proceedings of the IEEE Symposium Series on Computational Intelligence*. IEEE, 1606–1613.

[42] Shaolin Wang, Yi Mei, and Mengjie Zhang. 2019. Novel ensemble genetic programming hyper-heuristics for uncertain capacitated arc routing problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1093–1101.

[43] Shaolin Wang, Yi Mei, and Mengjie Zhang. 2020. A Multi-Objective Genetic Programming Hyper-Heuristic Approach to Uncertain Capacitated Arc Routing Problems. In *Proceedings of the IEEE Congress on Evolutionary Computation*. IEEE, 1–8.

[44] Thomas Weise, Alexandre Devert, and Ke Tang. 2012. A developmental solution to (dynamic) capacitated arc routing problems using genetic programming. In *Proceedings of the conference on Genetic and evolutionary computation*. ACM, 831–838.

[45] Sanne Wøhlk. 2008. A decade of capacitated arc routing. In *The vehicle routing problem: latest advances and new challenges*. Springer, 29–48.

[46] Fangfang Zhang, Yi Mei, Su Nguyen, Kay Chen Tan, and Mengjie Zhang. 2021. Multitask Genetic Programming-Based Generative Hyper-heuristics: A Case Study in Dynamic Scheduling. *IEEE Transactions on Cybernetics* (2021). Doi: 10.1109/TCYB.2021.3065340

[47] Fangfang Zhang, Yi Mei, Su Nguyen, and Mengjie Zhang. 2020. Collaborative Multi-fidelity Based Surrogate Models for Genetic Programming in Dynamic Flexible Job Shop Scheduling. *IEEE Transactions on Cybernetics* (2020). Doi: 10.1109/TCYB.2021.3050141

[48] Fangfang Zhang, Yi Mei, Su Nguyen, and Mengjie Zhang. 2021. Correlation Coefficient based Recombinative Guidance for Genetic Programming Hyper-heuristics in Dynamic Flexible Job Shop Scheduling. *IEEE Transactions on Evolutionary Computation* (2021). Doi:10.1109/TEVC.2021.3056143

[49] Fangfang Zhang, Yi Mei, Su Nguyen, and Mengjie Zhang. 2021. Evolving Scheduling Heuristics via Genetic Programming with Feature Selection in Dynamic Flexible Job Shop Scheduling. *IEEE Transactions on Cybernetics* 51, 4 (2021), 1797–1811.

[50] Fangfang Zhang, Yi Mei, Su Nguyen, Mengjie Zhang, and Kay Chen Tan. 2021. Surrogate-Assisted Evolutionary Multitask Genetic Programming for Dynamic Flexible Job Shop Scheduling. *IEEE Transactions on Evolutionary Computation* (2021). Doi:10.1109/TEVC.2021.3065707

[51] Fangfang Zhang, Yi Mei, and Mengjie Zhang. 2019. Evolving Dispatching Rules for Multi-objective Dynamic Flexible Job Shop Scheduling via Genetic Programming Hyper-heuristics. In *Proceedings of IEEE Congress on Evolutionary Computation*. IEEE, 1366–1373.

[52] Qingfu Zhang and Hui Li. 2007. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation* 11, 6 (2007), 712–731.